

Logistic Regression for Business Classification

A Comparative Study Across 10 Real-World Business Problems

Project Documentation / Report

Submitted by: Bharathi

Computer Science & Engineering, ATME College of Engineering, Mysuru

Training Program: LearnDepth — Machine Learning Track

1. Project Overview

This project applies Logistic Regression — a foundational supervised machine learning algorithm for binary classification — to 10 distinct, real-world-inspired business problems. Each problem uses its own dataset (1,000 records, 6 numeric features, a balanced binary target) and is carried through an identical, reproducible modelling pipeline: data inspection, preprocessing, baseline comparison, hyperparameter-tuned training, cross-validation, evaluation, and coefficient-level interpretation.

The 10 business problems covered are:

- **01. Customer Churn Risk** — Predict whether a subscriber will cancel their subscription.
- **02. Loan Approval Screening** — Predict whether a loan application should be approved.
- **03. Employee Attrition** — Predict whether an employee is likely to leave the company.
- **04. E-commerce Purchase Intent** — Predict whether a browsing session will end in a purchase.
- **05. Insurance Claim Flagging** — Predict whether a claim should be flagged for manual review.
- **06. Equipment Failure Warning** — Predict whether industrial equipment will fail soon.
- **07. Student Placement Eligibility** — Predict whether a student meets placement eligibility.
- **08. Email Priority Classification** — Predict whether an email should be marked high priority.
- **09. Delivery Delay Risk** — Predict whether an order will be delivered late.
- **10. Retail Return Risk** — Predict whether a retail order is likely to be returned.

2. Methodology

The same pipeline was applied to all 10 problems, implemented as a shared, reusable Python module so that results across problems are directly comparable (differences in score reflect genuine differences in class separability, not inconsistent methodology).

- **1. Data Inspection** — Checked shape, data types, missing values, duplicate rows, class balance, and feature correlations for every dataset before any modelling.
- **2. Preprocessing & Split** — Performed an 80/20 stratified train/test split **before** scaling. A StandardScaler was fit only on the training data and then applied to the test data, to avoid data leakage.
- **3. Baseline** — Trained a majority-class DummyClassifier for every problem, establishing the accuracy floor that a real model must exceed to be considered useful.
- **4. Model Training** — Trained Logistic Regression with hyperparameters tuned via GridSearchCV, searching over L1/L2 penalty and 8 values of the regularization strength C, scored on ROC-AUC.
- **5. Cross-Validation** — Ran 5-fold stratified cross-validation on the training data to check that performance was stable across folds, independent of the held-out test set.
- **6. Evaluation** — Measured accuracy, precision, recall, F1-score, ROC-AUC, and the full confusion matrix on the held-out test set for every problem.
- **7. Interpretation** — Converted standardized coefficients into odds ratios to explain, in plain terms, which features increase or decrease the predicted probability and by how much.

3. Results Summary

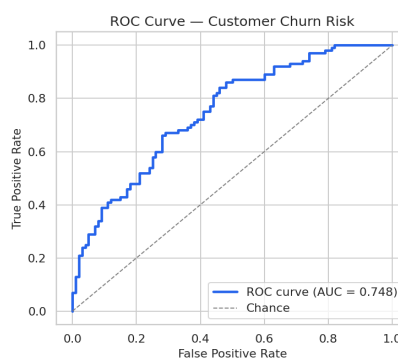
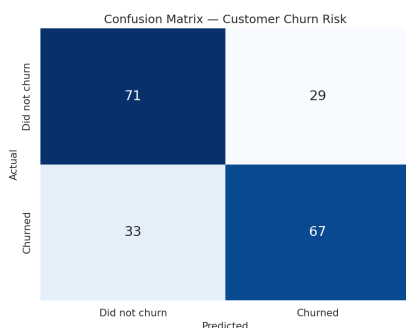
All 10 datasets were clean (no missing values or duplicates) and perfectly class-balanced (50/50), so the baseline accuracy is 50% in every case — any improvement above that is attributable entirely to the model and its features.

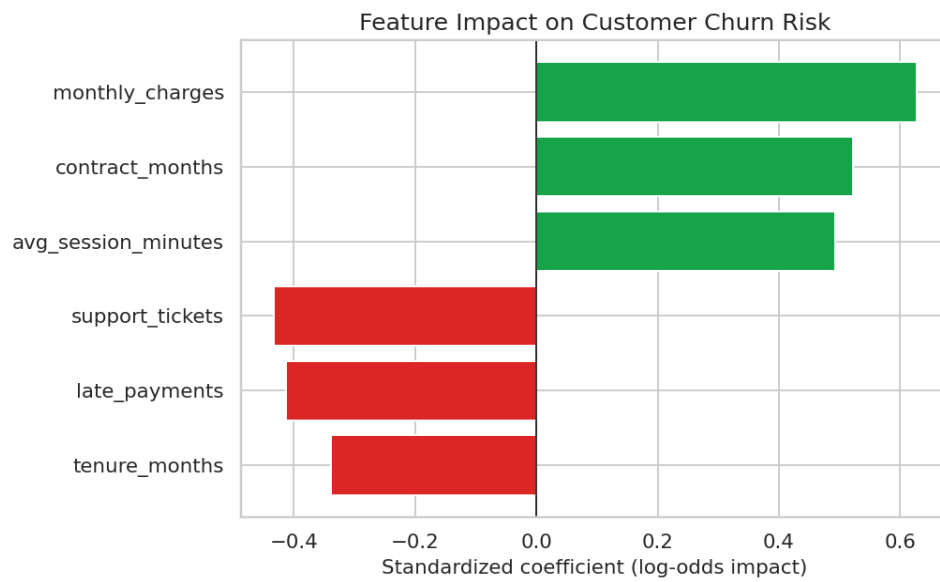
#	Problem	Accuracy	Precision	Recall	F1	ROC-AUC
01	Customer Churn Risk	69.0%	0.70	0.67	0.68	0.748
02	Loan Approval Screening	70.5%	0.69	0.75	0.72	0.782
03	Employee Attrition	66.0%	0.66	0.66	0.66	0.737
04	E-commerce Purchase Intent	72.5%	0.74	0.70	0.72	0.800
05	Insurance Claim Flagging	62.5%	0.63	0.60	0.62	0.666
06	Equipment Failure Warning	70.5%	0.73	0.66	0.69	0.776
07	Student Placement Eligibility	70.0%	0.69	0.73	0.71	0.787
08	Email Priority Classification	66.0%	0.65	0.69	0.67	0.734
09	Delivery Delay Risk	62.5%	0.61	0.68	0.64	0.715
10	Retail Return Risk	67.0%	0.65	0.72	0.69	0.731
Average		67.7%				0.748

Across all 10 problems, accuracy ranged from 62.5% to 72.5% and ROC-AUC ranged from 0.666 to 0.800, averaging 67.7% accuracy and 0.748 ROC-AUC. The strongest-performing model was **Problem 04 — E-commerce Purchase Intent** (ROC-AUC 0.800), while the weakest was **Problem 05 — Insurance Claim Flagging** (ROC-AUC 0.666). All 10 models beat their 50% baseline by a wide margin, confirming the features in every dataset carry genuine predictive signal.

Illustrative Example — Problem 01: Customer Churn Risk

As a representative example, the plots below show the confusion matrix, ROC curve, and standardized coefficient impact generated for the Customer Churn Risk model. Equivalent plots were generated for all 10 problems.





Confusion matrix, ROC curve (AUC = 0.748), and standardized coefficient impact for the Customer Churn Risk model.

4. What I Learned From This Project

Building this project end-to-end (across 10 problems rather than just one) required applying the same set of concepts repeatedly and rigorously. The key concepts learned are listed below.

Data Preparation

- Exploratory data analysis: checking missing values, duplicate rows, class balance, and correlation between features before modelling.
- Why the train/test split must be performed before scaling, and why the scaler must be fit only on training data — to avoid data leakage.
- Feature scaling with `StandardScaler` and why Logistic Regression, as a regularized linear model, is sensitive to feature scale.

Core Algorithm

- Logistic Regression fundamentals: the sigmoid function, log-odds, and the linear decision boundary in log-odds space.
- L1 (Lasso) vs. L2 (Ridge) regularization, and the role of the inverse-regularization-strength hyperparameter C .

Model Selection & Validation

- Hyperparameter tuning using `GridSearchCV` to systematically search a parameter grid.
- Stratified k-fold cross-validation, and why it gives a more reliable estimate of generalization than a single train/test split.
- The importance of a baseline model (a majority-class `DummyClassifier`) to know whether a model is actually adding predictive value.

Evaluation

- Evaluation metrics beyond accuracy — precision, recall, F1-score, and ROC-AUC — and when each one matters more than the others (e.g. recall vs. precision trade-offs when false negatives and false positives have different costs).
- Reading and interpreting a confusion matrix (true/false positives and negatives).
- Reading and interpreting an ROC curve.

Interpretability

- Interpreting model coefficients as odds ratios to explain, in business terms, which features drive a prediction and by how much — and why this interpretability is a key advantage of Logistic Regression over black-box models.

Software Practice

- Writing modular, reusable code: building one shared pipeline and reusing it across 10 datasets, instead of duplicating logic 10 times.
- Structuring a project for reproducibility (fixed random seeds, clear folder structure, auto-generated reports) so results can be regenerated exactly.

5. AI Usage Disclosure

I used Claude (Anthropic) as a coding and documentation assistant during this project. The table below breaks down which phases involved AI assistance and what it was used for.

Phase	How AI Was Used
Planning & Architecture	Discussed how to structure the project before writing code — decided on one shared pipeline module reused across all 10 datasets, instead of duplicating logic in 10 separate scripts.
Implementation	Generated the core pipeline (preprocessing, GridSearchCV tuning, cross-validation, evaluation) and the 10 per-problem scripts, which I then reviewed for correctness.
Visualization	Generated the confusion matrix, ROC curve, correlation heatmap, and coefficient-importance plots for each problem.
Debugging	Fixed plotting issues (e.g. chart titles being cut off) and resolved library version warnings during test runs.
Documentation	Drafted the per-problem written reports, the comparative summary report, and this documentation PDF, based on the actual computed results.

All AI-generated code and content was reviewed, tested, and understood before being included in the final submission. I can independently explain the pipeline, the concepts underlying each step, and the meaning of every result in this report.